UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
ERIK MAZZUCO
LETÍCIA MORO
LEONARDO HERRERO

ÁRVORES DIGITAIS NO ROTEAMENTO IPV4: DESEMPENHO DA PATRICIA E DA LC-TRIE NA BUSCA PELO PREFIXO MAIS LONGO

MEDIANEIRA
2026

ERIK MAZZUCO
LETÍCIA MORO
LEONARDO HERRERO

ÁRVORES DIGITAIS NO ROTEAMENTO IPV4: DESEMPENHO DA PATRICIA E DA LC-TRIE NA BUSCA PELO PREFIXO MAIS LONGO

Relatório do Projeto Prático 3 – Árvores Digitais [Patricia e LC-Trie], apresentado à disciplina de Árvores e Grafos do curso de Ciência da Computação da Universidade Tecnológica Federal do Paraná (UTFPR), Câmpus Medianeira, no período letivo 2026.2.

MEDIANEIRA
2026

RESUMO

Roteadores IPv4 escolhem a interface de saída de cada pacote pela rota de maior prefixo coincidente (longest prefix match, LPM), em uma tabela global com mais de um milhão de prefixos. Este trabalho implementa duas árvores digitais para essa tarefa, a Patricia (compressão de caminho) e a LC-Trie (compressão de caminho e de nível), e compara seu desempenho. As estruturas foram escritas em Go e validadas contra uma implementação de referência independente, com testes aleatórios e com a tabela completa. Foi usada a tabela BGP do projeto RouteViews de 1º de julho de 2026, com 1.101.804 rotas distintas e 16 interfaces. Os experimentos mediram tempo de consulta, profundidade de busca, memória, tempo de construção e custo de atualização, com três tipos de tráfego e quatro fatores de preenchimento da LC-Trie. Com fator de preenchimento 0,5, a LC-Trie foi de 3,5 a 4,7 vezes mais rápida que a Patricia nas consultas (de 522,7 para 112,1 ns por consulta no tráfego que casa com a tabela), visitou cerca de quatro vezes menos nós e ocupou 16% menos memória estrutural, com tempo de construção equivalente. Em contrapartida, a LC-Trie é estática: incluir uma única rota exige reconstruir a árvore (cerca de 475 ms), enquanto a Patricia insere uma rota em cerca de 655 ns. A Patricia é, portanto, a melhor escolha quando a tabela muda com frequência ou quando a simplicidade da implementação pesa mais que a taxa de consulta.

Palavras-chave: árvores digitais; Patricia; LC-Trie; roteamento IP; prefixo mais longo.

ABSTRACT

IPv4 routers pick the output interface of each packet using the longest prefix match (LPM) over a global table with more than one million prefixes. This work implements two digital trees for this task, the Patricia trie (path compression) and the LC-trie (path and level compression), and compares their performance. Both were written in Go and validated against an independent reference implementation, using random tests and the full table. The RouteViews BGP table of July 1st, 2026 was used, with 1,101,804 distinct routes and 16 interfaces. The experiments measured lookup time, search depth, memory, build time and update cost, using three kinds of traffic and four LC-trie fill factors. With fill factor 0.5, the LC-trie was 3.5 to 4.7 times faster than the Patricia trie at lookups (from 522.7 to 112.1 ns per lookup on traffic that matches the table), visited about four times fewer nodes and used 16% less structural memory, with equivalent build time. On the other hand, the LC-trie is static: adding a single route requires rebuilding the tree (about 475 ms), whereas the Patricia trie inserts a route in about 655 ns. Patricia is therefore the best choice when the table changes often or when implementation simplicity matters more than lookup rate.

Keywords: digital trees; Patricia; LC-trie; IP routing; longest prefix match.

SUMÁRIO

1 INTRODUÇÃO....6
1.1 Objetivos....6
1.2 Uso de ferramentas de IA....6
2 FUNDAMENTAÇÃO TEÓRICA....7
2.1 Prefixos como cadeias de bits....7
2.2 Patricia....7
2.3 LC-Trie....7
3 IMPLEMENTAÇÃO....8
3.1 Formato dos arquivos....8
3.2 Detalhes da LC-Trie....8
3.3 Patricia....9
4 METODOLOGIA....9
4.1 Dados....9
4.2 Tráfego de consulta....9
4.3 Medições....10
4.4 Validação....10
5 RESULTADOS....11
5.1 Construção e memória....11
5.2 Consulta....11
5.3 Atualização....12
6 DISCUSSÃO....13
6.1 Quanto a LC-Trie ganha em velocidade....13
6.2 Quanto isso custa em memória e em tempo de construção....13
6.3 Quando a Patricia é a melhor escolha....13
6.4 Limitações....14
7 CONCLUSÃO....14
7.1 Reprodução....14
REFERÊNCIAS....15

1 INTRODUÇÃO

Um roteador IPv4 recebe pacotes e precisa decidir, para cada endereço de destino, por qual interface encaminhá-lo. As rotas são blocos CIDR, como 10.1.0.0/16, em que os 16 primeiros bits são fixos e o restante é livre. Um mesmo endereço costuma casar com várias rotas ao mesmo tempo, e vale a mais específica, ou seja, a de prefixo mais longo (longest prefix match, LPM). A tabela global da internet tem na ordem de um milhão de prefixos IPv4 e a decisão precisa ser tomada em nanossegundos, o que exige uma estrutura de dados adequada.

Árvores digitais são a solução clássica: a Patricia foi usada na tabela de rotas do BSD (SKLOWER, 1991) e a tabela IPv4 do kernel Linux (fib_trie) se baseia na LC-Trie (NILSSON; KARLSSON, 1999). Este relatório descreve a implementação das duas estruturas e responde, com números, a três perguntas do enunciado do Projeto Prático 3: quanto a LC-Trie ganha em velocidade de consulta sobre a Patricia; quanto isso custa em memória e em tempo de construção; e quando a Patricia é a melhor escolha.

1.1 Objetivos

- Representar endereços e prefixos como cadeias de bits e manipulá-los com operações bit a bit.
- Implementar a Patricia e a LC-Trie e usá-las em um simulador de encaminhamento de pacotes.
- Medir e explicar o compromisso entre profundidade, uso de memória e facilidade de atualização.

1.2 Uso de ferramentas de IA

Conforme permitido pelas diretrizes do projeto, a implementação e a primeira versão deste relatório foram desenvolvidas com apoio de uma ferramenta de IA (Claude Code). O código foi validado pelos testes automatizados descritos na Seção 4.4, e todos os números apresentados foram obtidos pela execução do programa entregue.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Prefixos como cadeias de bits

Um endereço IPv4 é um inteiro de 32 bits. Um prefixo a/k casa com um endereço x quando os k primeiros bits são iguais, o que se testa com deslocamento: (x XOR a) >> (32 − k) = 0. O prefixo /0 (rota padrão) casa com qualquer endereço. Como prefixos podem ser prefixos uns dos outros, uma busca por LPM deve guardar a melhor rota vista no caminho e continuar descendo.

2.2 Patricia

A Patricia (MORRISON, 1968) é uma árvore binária em que sequências de bits sem bifurcação são comprimidas: cada nó pula diretamente para o próximo bit em que há decisão. Nesta implementação, cada nó guarda o prefixo completo que o identifica (chave e comprimento), um indicador de que há rota associada e dois filhos. Nós de junção, criados quando dois prefixos divergem em um bit, não têm rota e sempre têm dois filhos, de modo que a árvore tem no máximo cerca de 2n nós para n rotas. A inserção desce comparando o prefixo comum e termina em um de quatro casos: prefixo repetido, nó atual é prefixo da nova rota (desce), nova rota é prefixo do nó atual (entra acima dele) ou divergência (cria nó de junção). A inserção custa O(32) e permite atualizações incrementais.

2.3 LC-Trie

A LC-Trie (NILSSON; KARLSSON, 1999) acrescenta compressão de nível: um nó interno testa b bits de uma só vez e tem 2^b filhos, desde que eles sejam suficientemente povoados. Cada nó guarda três campos: branch (quantos bits testar), skip (quantos bits pular, por compressão de caminho) e um ponteiro para os filhos, que ficam contíguos em um vetor. Isso reduz muito a profundidade e os acessos à memória por consulta. Um nó aumenta o branch enquanto pelo menos fill · 2^b dos filhos forem não vazios; o parâmetro fill é o fator de preenchimento e controla o compromisso entre profundidade e memória.

A construção exige que o conjunto de strings seja livre de prefixos, o que não vale para tabelas de rotas. A solução original separa as rotas que são prefixo de outras em um vetor de prefixos (pv) e deixa as demais, ordenadas, em um vetor base. Cada

entrada guarda um ponteiro pre para a rota interna mais longa que a cobre. A consulta desce a trie até uma folha do vetor base, testa se ela casa e, se não casar, percorre a cadeia pre, da mais longa para a mais curta.

3 IMPLEMENTAÇÃO

O código está em Go, sem dependências externas, no diretório src. O programa tem três comandos: rotear (lê rotas.csv e consultas.txt e grava saida.txt), gerar (cria consultas sintéticas) e bench (executa os experimentos). A Tabela 1 resume os arquivos.

Tabela 1 – Organização do código-fonte

Arquivo	Conteúdo
ip.go	Leitura de rotas e consultas, conversão de endereços, interface Lookuper
patricia.go	Patricia: inserção, busca LPM, contagem de nós e de memória
lctrie.go	LC-Trie: construção com fator de preenchimento, busca, memória
ref.go	Referência: uma tabela hash por tamanho de prefixo (oráculo e linha de base)
main.go / bench.go	Comandos rotear, gerar e bench
trees_test.go	Testes automatizados

Fonte: Autoria própria (2026).

3.1 Formato dos arquivos

A entrada rotas.csv tem uma rota por linha no formato prefixo,tamanho,interface, com cabeçalho opcional. Bits de host do prefixo são zerados e duplicatas mantêm a última ocorrência (a tabela usada não tem duplicatas). Cada linha de saida.txt tem a forma endereço interface prefixo_vencedor, por exemplo:

```
10.1.9.9 eth2 10.1.0.0/16
```

Se nenhuma rota casar, nem a padrão, a linha é "endereço descartado". Com a tabela de exemplo do enunciado, as seis consultas dão exatamente os resultados esperados (eth3, eth2, eth1, eth5, eth4 e eth0), o que também é verificado por teste automatizado.

3.2 Detalhes da LC-Trie

O nó é um inteiro de 32 bits com branch (5 bits), skip (5 bits) e endereço (22 bits), como no artigo original. O branch máximo é 24 bits, e todos os prefixos de um intervalo precisam ter ao menos os bits testados, o que garante que a busca nunca

olhe para além do fim de um prefixo. Um ponto precisou de solução própria: com fill < 1, um filho do nó pode ficar vazio, e apontá-lo para uma folha vizinha faria uma rota interna que cobre aquela região ser perdida. A solução adotada cria, para cada filho vazio, uma folha sintética que nunca casa sozinha, mas cuja cadeia pre aponta para a rota interna que cobre a região (por exemplo, a rota padrão). Essas folhas aparecem na coluna de composição da Tabela 2 e entram no custo de memória.

3.3 Patricia

Os nós têm 16 bytes (chave de 32 bits, dois índices de filhos de 32 bits, interface, comprimento e indicador de rota), e os filhos são índices em um vetor, o que dá localidade melhor que ponteiros e permite contar a memória com exatidão. Como a busca testa o prefixo inteiro do nó, não é preciso fazer retrocesso (backtracking) como na Patricia original do BSD.

4 METODOLOGIA

4.1 Dados

Foi usada a tabela BGP do RouteViews de 01/07/2026 00:00 (arquivo rib.20260701.0000.rotas_ipv4.csv): 1.101.804 rotas distintas, 16 interfaces (eth0 a eth15) e rota padrão 0.0.0.0/0. Os prefixos vão de /8 a /32 (além da rota padrão), com forte predominância de /24 (700.417 rotas, 63,6%), seguidos de /22 (116.248) e /23 (114.328).

4.2 Tráfego de consulta

Como o enunciado não fornece consultas, foram geradas 2.000.000 de consultas por trace, com semente fixa: (a) rotas: sorteia uma rota da tabela e um endereço dentro dela, o que representa tráfego que casa com prefixos específicos; (b) aleatório: 32 bits uniformes, o que representa tráfego espalhado pelo espaço de endereços e tende a casar com prefixos curtos; (c) misto: metade de cada. As consultas ficam em memória durante a medição, de modo que não se mede entrada e saída.

4.3 Medições

Para cada estrutura, o tempo de construção é a mediana de 7 construções a partir das rotas já carregadas (a LC-Trie inclui a ordenação). O tempo de consulta é a mediana de 7 passadas sobre o trace, em ns por consulta, em uma única thread. A memória estrutural é calculada pelo tamanho dos vetores (Patricia: 16 bytes por nó; LC-Trie: 4 bytes por nó da trie e 16 por entrada de base e de pv), e a coluna de heap mede o crescimento do heap do Go após uma coleta de lixo. A profundidade é o número de nós visitados em 200.000 consultas. O ambiente foi Go 1.25.2, Windows 11, amd64, 8 CPUs, em uma única execução do programa.

4.4 Validação

Uma implementação independente de LPM, com uma tabela hash por comprimento de prefixo, serve de oráculo. Os testes automatizados (go test) comparam as duas árvores com ela em 300 tabelas aleatórias com prefixos muito aninhados, com e sem rota padrão, com fatores de preenchimento 1, 0,5 e 0,25, e na tabela BGP completa com 600.000 consultas. Além disso, o benchmark compara o checksum dos resultados de cada estrutura com o do oráculo em todos os traces. Todos os testes passaram, e as saídas da Patricia e da LC-Trie para 100.000 consultas são idênticas byte a byte.

5 RESULTADOS

5.1 Construção e memória

Tabela 2 – Construção e memória com 1.101.804 rotas

Estrutura	Build (ms)	Estrutural (MB)	Heap (MB)	Composição
Patricia	116,5	30,9	33,6	2.023.207 nós (921.403 de junção)
LC-Trie f=1,00	100,1	22,3	24,6	trie 1.443.123; base 997.665; pv 104.139
LC-Trie f=0,75	95,9	23,7	28,5	base 1.093.582 (95.917 sintéticas)
LC-Trie f=0,50	104,6	25,9	28,5	base 1.213.916 (216.251 sintéticas)
LC-Trie f=0,25	115,7	28,3	33,2	base 1.342.660 (344.995 sintéticas)
Hash por tamanho	83,0	–	15,0	linha de base (maps do Go)

Fonte: Autoria própria (2026). Execução do comando bench.

A Patricia precisa de 2,02 milhões de nós para 1,10 milhão de rotas, pois 921 mil são nós de junção. A LC-Trie guarda as rotas em menos entradas e usa nós de apenas 4 bytes, porque a trie de alto grau substitui boa parte dos nós de junção por um vetor indexado; mesmo com fator de preenchimento 0,25 e 345 mil folhas sintéticas, ocupa menos memória estrutural que a Patricia. O tempo de construção das duas é da mesma ordem (de 96 a 117 ms), apesar de a LC-Trie precisar ordenar as rotas.

5.2 Consulta

Tabela 3 – Tempo de consulta (ns por consulta, mediana) e profundidade média (nós visitados)

Estrutura	Rotas	Aleatório	Misto	Prof. rotas	Prof. aleat.
Patricia	522,7	224,5	401,7	23,26	14,96
LC-Trie f=1,00	121,8	95,6	117,8	7,93	6,33
LC-Trie f=0,75	110,4	65,1	94,7	5,85	4,11
LC-Trie f=0,50	112,1	63,9	94,1	5,69	3,98
LC-Trie f=0,25	110,8	62,4	98,5	5,64	3,94
Hash por tamanho	232,9	466,4	363,2	9,89	19,67

Fonte: Autoria própria (2026). Todas as estruturas produziram resultados idênticos ao oráculo.

Tabela 4 – Ganho da LC-Trie sobre a Patricia (razão entre os tempos de consulta)

Estrutura	Rotas	Aleatório	Misto
LC-Trie f=1,00	4,29x	2,35x	3,41x
LC-Trie f=0,75	4,74x	3,45x	4,24x
LC-Trie f=0,50	4,66x	3,51x	4,27x
LC-Trie f=0,25	4,72x	3,60x	4,08x

Fonte: Autoria própria (2026).

A LC-Trie foi mais rápida em todos os cenários. Com f=0,5, o ganho foi de 4,66x no tráfego que casa com a tabela, 3,51x no aleatório e 4,27x no misto. A explicação está na profundidade: no tráfego de rotas, a Patricia visita em média 23,3 nós por consulta (máximo 30) contra 5,7 da LC-Trie, e cada nó visitado é um acesso à memória em uma tabela de 30 MB, provavelmente maior que a cache do processador. O tempo da Patricia é maior no trace de rotas (522,7 ns) que no aleatório (224,5 ns) porque endereços aleatórios casam com prefixos curtos, nos níveis altos da árvore, enquanto endereços dentro de rotas específicas descem até o nível /24.

O fator de preenchimento tem efeito claro entre 1,0 e 0,75 (ganho de 2,35x para 3,45x no tráfego aleatório) e pouco efeito abaixo disso: de 0,75 para 0,25 a profundidade quase não muda, mas a memória estrutural cresce de 23,7 para 28,3 MB. O valor 0,5 é um bom ponto de equilíbrio nesta tabela. A linha de base com hash por tamanho oscila de 233 a 466 ns, pois o custo depende de quantos comprimentos precisam ser testados antes do acerto, e perde para a LC-Trie em todos os traces.

5.3 Atualização

Foram inseridas 10.000 rotas /24 novas, com prefixos ainda não presentes. Na Patricia, as inserções incrementais levaram 6,55 ms, ou cerca de 655 ns por rota. A LC-Trie não admite inserção: a estrutura depende de uma ordenação global e do cálculo do branch de cada nó a partir de todos os prefixos do intervalo, então qualquer mudança exige reconstruir a árvore, o que levou 474,8 ms (f=0,5). Uma única atualização isolada custa, portanto, cerca de 725.000 vezes mais na LC-Trie que na Patricia nesta tabela. Mesmo agrupando as mudanças em lotes de 10.000 rotas, a reconstrução (47,5 µs por rota) é cerca de 72 vezes mais cara que a inserção incremental.

6 DISCUSSÃO

6.1 Quanto a LC-Trie ganha em velocidade

Entre 3,5 e 4,7 vezes, com f=0,5, conforme o tipo de tráfego: de 522,7 para 112,1 ns (4,66x) no tráfego que casa com a tabela, de 224,5 para 63,9 ns (3,51x) no aleatório e de 401,7 para 94,1 ns (4,27x) no misto. Isso equivale a passar de 1,9 para 8,9 milhões de consultas por segundo por thread no primeiro caso. A causa dominante é a redução de profundidade (de 23,3 para 5,7 nós visitados), que reduz acessos à memória.

6.2 Quanto isso custa em memória e em tempo de construção

Neste experimento, o custo foi nulo ou negativo: com f=0,5, a LC-Trie ocupa 25,9 MB estruturais contra 30,9 MB da Patricia (16% menos) e constrói em 104,6 ms contra 116,5 ms. Mesmo com f=0,25 a memória é 8% menor. O resultado difere da expectativa de que a compressão de nível gasta mais memória, e é explicado pelo nó de 4 bytes da LC-Trie e pelo nó de 16 bytes que a Patricia desta implementação usa para cada rota e cada junção. Uma Patricia com nós mais compactos reduziria essa vantagem, de modo que a conclusão vale para as implementações e para a tabela aqui usadas. O custo real da LC-Trie não está na memória nem na construção, mas na atualização (Seção 5.3).

6.3 Quando a Patricia é a melhor escolha

- Tabelas dinâmicas: com atualizações BGP frequentes, a Patricia insere uma rota em cerca de 655 ns, ao passo que a LC-Trie precisa ser reconstruída em cerca de 475 ms (ou mantida em duas cópias, com troca atômica).
- Tabelas pequenas ou consultas raras: se a tabela cabe na cache, a profundidade deixa de ser dominada por acessos à memória e o ganho da LC-Trie tende a diminuir, enquanto a simplicidade da Patricia permanece. Esse efeito não foi medido neste trabalho.
- Simplicidade de implementação e correção: a Patricia tem um único tipo de nó e quatro casos de inserção. A LC-Trie exige separar prefixos internos, tratar filhos vazios e escolher o fator de preenchimento; um erro nesses

pontos devolve rotas erradas, e foi necessário tratá-los explicitamente neste trabalho.

6.4 Limitações

Os tempos vêm de uma única execução em um computador com Windows, e valores absolutos variam com o processador e a cache; as razões entre estruturas são mais confiáveis que os números absolutos. A linguagem Go inclui verificação de limites de vetores, o que acrescenta custo em todas as estruturas. O tráfego é sintético, sem a distribuição de popularidade de destinos de uma rede real, que favoreceria ambas as estruturas pela cache. A remoção de rotas não foi implementada.

7 CONCLUSÃO

A Patricia e a LC-Trie foram implementadas e validadas contra uma referência independente, e produzem o mesmo resultado em toda a tabela BGP de 1,1 milhão de rotas. A LC-Trie foi de 3,5 a 4,7 vezes mais rápida na consulta, visitou cerca de quatro vezes menos nós e, nesta implementação, usou menos memória e construiu em tempo equivalente. Seu custo está na rigidez: qualquer atualização exige reconstrução completa, cerca de 475 ms contra 655 ns por inserção na Patricia. A escolha entre as duas depende, portanto, da taxa de mudança da tabela em relação à taxa de consultas: a LC-Trie para uma tabela de encaminhamento quase estática e muito consultada, a Patricia quando as atualizações são frequentes ou a estrutura precisa ser simples.

7.1 Reprodução

Os comandos, a partir do diretório src e com o arquivo de rotas BGP copiado como rotas.csv, são:

```
go test ./...
go build -o arvoresdigitais.exe .
arvoresdigitais.exe bench  -rotas rotas.csv
arvoresdigitais.exe gerar  -rotas rotas.csv -saida consultas.txt
arvoresdigitais.exe rotear -rotas rotas.csv -consultas consultas.txt
                           -saida saida.txt -arvore lctrie
```

REFERÊNCIAS

FULLER, V.; LI, T. **RFC 4632**: Classless Inter-domain Routing (CIDR). IETF, 2006.

MORRISON, D. R. PATRICIA: Practical Algorithm To Retrieve Information Coded in Alphanumeric. **Journal of the ACM**, v. 15, n. 4, 1968.

NILSSON, S.; KARLSSON, G. IP-address lookup using LC-tries. **IEEE Journal on Selected Areas in Communications**, v. 17, n. 6, 1999.

RUIZ-SÁNCHEZ, M. A.; BIERSACK, E. W.; DABBOUS, W. Survey and taxonomy of IP address lookup algorithms. **IEEE Network**, v. 15, n. 2, 2001.

SKLOWER, K. A tree-based packet routing table for Berkeley Unix. In: **USENIX Winter Conference**, 1991.

UNIVERSITY OF OREGON. **Projeto RouteViews**. Disponível em: routeviews.org. Acesso em: jul. 2026.

VARGHESE, G. **Network Algorithmics**. Morgan Kaufmann, 2005.